Date:-1/10/2026

Week-1

Genetic Algorithm for Optimization Problems

Pseudocode

BEGIN

1. Define fitness function:

    $f(x) = x^2$

2. Initialize parameters:

    POP_SIZE = 6

    MAX_GEN = 50

    CROSSOVER_RATE = 0.8

    MUTATION_RATE = 0.01

3. Generate initial population randomly:

    POPULATION = Random solutions between 0 and 31

4. FOR generation = 1 TO MAX_GEN:

    a. Evaluate Fitness:

        Calculate fitness = f(x)

        for each individual in POPULATION

    b. Selection:

        Select parents based on their fitness

        using Roulette Wheel Selection

    c. Crossover:

        Select two parents

        Generate offspring by exchanging genes

    d. Mutation:

        Randomly change genes of offspring

        based on MUTATION_RATE

    e. Create New Population:

Replace old population with offspring

f. Find Best Solution:

Identify individual with highest fitness

Store the best solution

5. OUTPUT:

Best solution (x)

Maximum fitness value (f(x))

END

Python implementation: Genetic Algorithm for Robot Path Planning

```
import random
import numpy as np
import matplotlib.pyplot as plt

# Environment
START = (1, 1)
GOAL = (18, 18)
OBSTACLES = [(5, 5), (6, 6), (7, 7), (10, 10), (12, 12)]

# GA Parameters
POP_SIZE = 20
GENERATIONS = 50
CROSSOVER_RATE = 0.8
MUTATION_RATE = 0.2

# Generate random path
def create_path():
    return [[random.randint(2, 17),
            random.randint(2, 17)] for _ in range(4)]
```

```
# Fitness function
def fitness(path):
    points = [START] + path + [GOAL]
    length = 0
    penalty = 0
    smoothness = 0
    for i in range(len(points)-1):
        length += np.linalg.norm(
            np.array(points[i+1]) - points[i])
    for p in path:
        if tuple(p) in OBSTACLES:
            penalty += 1000
    for i in range(1, len(points)-1):
        a = np.array(points[i]) - points[i-1]
        b = np.array(points[i+1]) - points[i]
        smoothness += np.linalg.norm(a-b)
    return length + 7*smoothness + penalty
# Selection
def selection(population):
    return min(population, key=fitness)

# Crossover
def crossover(p1, p2):
    if random.random() < CROSSOVER_RATE:
        point = random.randint(1, 3)
        return p1[:point] + p2[point:]
    return p1.copy()

# Mutation
def mutation(path):
```

```
    for p in path:
        if random.random() < MUTATION_RATE:
            p[0] = random.randint(2, 17)
            p[1] = random.randint(2, 17)
    return path

# Genetic Algorithm
population = [create_path() for _ in range(POP_SIZE)]
for gen in range(GENERATIONS):
    population.sort(key=fitness)
    new_population = [population[0]]
    while len(new_population) < POP_SIZE:
        p1 = selection(population)
        p2 = random.choice(population)
        child = crossover(p1, p2)
        child = mutation(child)
        new_population.append(child)
    population = new_population
best = min(population, key=fitness)
print("Best Path:", best)
print("Minimum Cost:", fitness(best))

# Display path
points = [START] + best + [GOAL]
x, y = zip(*points)
plt.plot(x, y, 'b-o', label="Robot Path")
for ox, oy in OBSTACLES:
    plt.scatter(ox, oy, color='red', marker='s')
plt.scatter(*START, color='green', label="Start")
plt.scatter(*GOAL, color='orange', label="Goal")
```

```
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.title("Genetic Algorithm Path Planning")
plt.grid()
plt.legend()
plt.show()
```

⇨ OUTPUT

```
Best Path: [[4, 5], [17, 11], [16, 12], [13, 15]]
Minimum Cost: 282.05337150236585
```

